

МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра МО ЭВМ

Курсовая РАБОТА
по дисциплине «Введение в нереляционные базы данных»
Тема: Инструмент для класстеризации данных прокторинга.

	_____	Вердин К.К.
	_____	Жаворонок Д.Н.
Студенты гр. 3344	_____	Бубякина Ю.В.
Преподаватель	_____	Заславский М.М.

Санкт-Петербург

2026

ЗАДАНИЕ

на курсовую работу «ИС для Ателье»

Студенты

Вердин К.К.

Бубякина Ю.В.

Жаворонок Д.Н.

Группа 3344

Тема работы: Инструмент для класстеризации данных прокторинга

Исходные данные:

Задача - создать веб-приложение для анализа экзаменационных сессий, где будут храниться логи Moodle, действия пользователей на веб-странице, данные распознавания лиц и результаты OCR по скринкастам. В приложении должна быть возможность загружать разнородные данные, объединять их по сессиям и студентам, фильтровать и агрегировать информацию, строить статистику, запускать несколько алгоритмов кластеризации, визуализировать кластеры сессий и выявлять аномальное поведение.

В качестве базы данных использовать MongoDB.

Содержание пояснительной записки:

«Содержание»,

«Введение»,

«Заключение»,

«Список использованных источников»

Предполагаемый объем пояснительной записки:

Не менее 61 страниц.

Дата выдачи задания: 10.02.2026

Дата сдачи реферата: 02.06.2026

Дата защиты реферата: 02.06.2026

	_____	Вердин К.К.
	_____	Жаворонок Д.Н.
	_____	Бубякина Ю.В.
Преподаватель	_____	Заславский М.М.

Аннотация

В рамках данного курса необходимо было разработать web-приложение в команде на одну из поставленных тем. Была выбрана реализация «Инструмента для класстеризации данных прокторинга» на базе нереляционной СУБД MongoDB. Исходный код представлен по ссылке <https://github.com/moevm/nsql1h26-proct>.

содержание

	Введение	6
1	Сценарии использования	8
1.1.	Макет UI	8
1.2.	Сценарии использования для задачи	8
1.3.	Сравнение количества операций чтения/записи	9
2.	Модель данных	10
2.1.	Нереляционная модель данных	10
2.2.	Аналог модели данных для SQL СУБД	32
2.3.	Сравнение моделей	50
3.	Разработанное приложение	54
3.1	Краткое описание	54
3.2	Используемые технологии	54
3.3	Снимки экрана приложения	54
	Заключение	59
	Список использованных источников	61
	Приложение А. Документация по сборке и развёртыванию	62

Введение

Актуальность

Актуальность разработки информационной системы для анализа экзаменационных сессий обусловлена необходимостью автоматизации обработки разнородных данных, связанных с прохождением онлайн-экзаменов. Использование цифрового решения позволяет централизованно хранить логи Moodle, действия пользователей на веб-странице, результаты распознавания лиц и OCR по скринкастам, а также объединять эти данные по студентам и экзаменационным сессиям.

Многие процессы анализа прокторинговых данных требуют ручной проверки и сопоставления информации из разных источников, что увеличивает риск ошибок, затрудняет поиск подозрительных ситуаций и замедляет принятие решений. Разработка специализированной информационной системы решает проблему разрозненного хранения данных, упрощает фильтрацию и агрегацию информации, позволяет применять алгоритмы кластеризации для выявления групп схожих сессий и обнаружения аномального поведения студентов.

Постановка задачи

Создать веб-приложение для кластеризации данных прокторинга, с ролями админа и преподавателя.

Предлагаемое решение

Необходимо реализовать веб-приложение для анализа экзаменационных сессий, использующее MongoDB в качестве основной базы данных для хранения студентов, сессий, логов Moodle, действий пользователей, данных OCR и результатов распознавания лиц. Для построения графического интерфейса сайта используется React, а для создания серверной части приложения используется Node.js и Express. Поднятие всего сервиса должно осуществляться через Docker Compose.

Качественные требования к решению

Должен поддерживаться массовый импорт и экспорт данных. В приложении должна быть возможность загружать разнородные данные

экзаменационных сессий, включая логи Moodle, действия пользователей, результаты OCR и данные распознавания лиц. Система должна хранить всю промежуточную информацию, связывать данные с конкретными студентами и сессиями, предоставлять фильтрацию, агрегацию и построение статистики.

Также в приложении должны быть реализованы несколько алгоритмов кластеризации, позволяющие группировать похожие экзаменационные сессии, визуализировать полученные кластеры и выявлять аномальное поведение студентов.

1. Сценарии использования

1.1. Макет UI

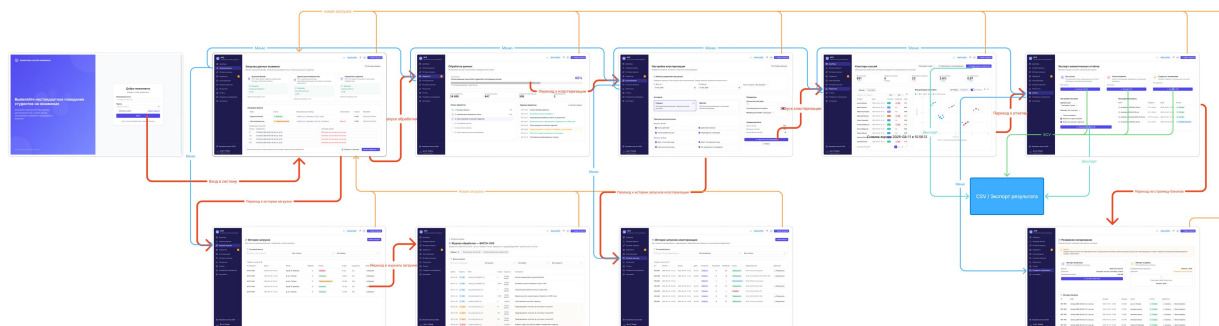


Рисунок 1 – Все экраны приложения

1.2. Сценарии использования

Импорт данных

Сценарий использования для импорта данных:

Вход в систему -> Вкладка «Загрузка данных» -> Выбор файлов на диске -> Загрузка логов Moodle, данных OCR, данных распознавания лиц и справочника студентов -> Проверка структуры данных на сервере -> Отображение статуса загрузки -> Просмотр проблемных строк и несопоставленных студентов -> Уведомление пользователя о результате операции.

Представление данных

Сценарий использования для представления данных:

Вход в систему -> Переход к разделам «Студенты», «Сессии», «События», «История загрузок» или «История запусков» -> Просмотр данных в табличном виде -> Использование фильтров и серверной пагинации -> Открытие карточки конкретной записи -> Просмотр подробной информации о студенте, сессии, событии, загрузке или запуске кластеризации.

Для администратора дополнительно доступны разделы управления пользователями, вузами, аудитом действий и резервным копированием базы данных

Обработка данных

Сценарий использования для обработки данных:

Вход в систему -> Вкладка «Загрузка данных» -> Нажатие кнопки «Начать

обработку» -> Переход на экран «Обработка» -> Выполнение этапов проверки файлов, нормализации данных, сопоставления студентов, построения сессий и вычисления метрик -> Отображение прогресса и журнала обработки -> Переход к подробному журналу загрузки при наличии ошибок.

Анализ данных

Сценарий использования для анализа данных:

Вход в систему -> Вкладка «Кластеризация» -> Выбор подмножества данных с помощью фильтров -> Выбор алгоритма кластеризации -> Настройка параметров и метрик -> Запуск кластеризации -> Переход на экран «Результаты» -> Просмотр кластеров, списка сессий, аномалий и визуализации распределения сессий.

Дополнительно пользователь может перейти в раздел «Статистика», выбрать сущность, задать многокритериальный фильтр, выбрать атрибуты по осям X и Y и получить диаграмму с количественным распределением данных.

Экспорт данных

Сценарий использования для экспорта данных:

Вход в систему -> Вкладка «Отчёты» -> Выбор типа выгрузки: все сессии, только аномалии или данные по студентам -> Настройка параметров отчёта -> Нажатие кнопки экспорта -> Формирование файла в формате CSV или JSON -> Скачивание результата на диск.

Резервное копирование

Сценарий использования для резервного копирования:

Вход в систему под администратором -> Вкладка «Резервное копирование» -> Выбор действия «Экспорт всей базы» или «Импорт из файла» -> Для экспорта система формирует полный дамп базы данных -> Для импорта пользователь выбирает файл бэкапа, подтверждает восстановление, после чего система восстанавливает данные из резервной копии.

1.3. Сравнение количества операций чтения/записи

Для разработанного приложения преобладающими являются операции чтения данных. Пользователи значительно чаще просматривают уже

существующие кластеры, выполняют поиск, фильтрацию, просматривают логи и статистику, чем создают или изменяют данные. Практически любое действие в интерфейсе сопровождается чтением информации из базы данных: загрузкой списка кластеров, студентов, сессий отображением содержимого.

2. Модель данных

2.1. Нереляционная СУБД

Графическое представление

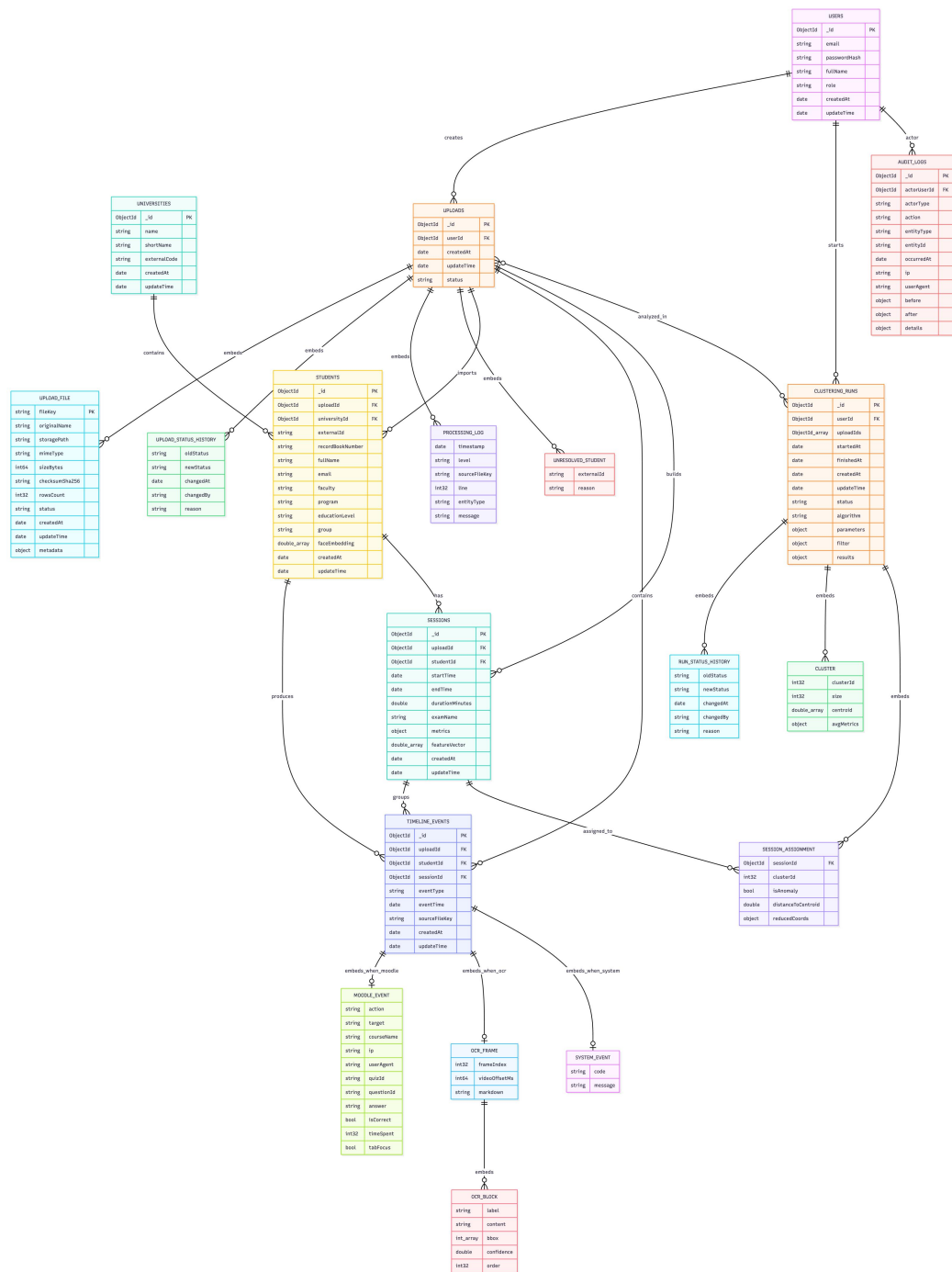


Рисунок 2 – Графическое представление NoSQL базы данных

Описание назначений коллекций, типов данных и сущностей

коллекций: users, universities, uploads, students, timeline_events, sessions, clustering_runs, audit_logs.

Коллекция User содержит информацию о пользователях системы: преподавателях и администраторах:

- `_id` - уникальный идентификатор пользователя (objectId, 12 Байт)
- `email` - электронная почта пользователя (обязательное, string, до 120 Байт)
- `passwordHash` - хеш пароля пользователя (обязательное, string, 60 Байт)
- `fullName` - полное имя пользователя (обязательное, string, до 120 Байт)
- `role` - роль пользователя (обязательное, string, до 10 Байт): admin, teacher
- `createdAt` - дата создания учетной записи (date, 8 Байт)
- `updateTime` - дата последнего изменения учетной записи (date, 8 Байт)

Коллекция uploads содержит справочник вузов и учебных организаций:

- `_id` - уникальный идентификатор вуза (objectId, 12 Байт)
- `name` - полное название вуза (обязательное, string, до 200 Байт)
- `shortName` - краткое название (string, до 50 Байт)
- `externalCode` - код вуза во внешней системе (string, до 30 Байт)
- `createdAt` - дата создания записи (date, 8 Байт)
- `updateTime` - дата последнего изменения записи (date, 8 Байт)

Коллекция содержит пакеты загрузки, метаданные исходных файлов, журнал обработки и историю смены статусов. Содержимое исходных файлов не хранится внутри документа: в MongoDB фиксируются метаданные и путь к файлу в файловом или объектном хранилище.

- `_id` - уникальный идентификатор загрузки (objectId, 12 Байт)
- `userId` - идентификатор пользователя-автора (обязательное, objectId, 12 Байт)
- `createdAt` - дата создания загрузки (date, 8 Байт)
- `updateTime` - дата последнего изменения загрузки (date, 8 Байт)

- status - текущий статус обработки (обязательное, string, до 12 Байт): pending, processing, done, error
- files - map исходных файлов, где ключом является тип или стабильный идентификатор файла:
 - originalName - исходное имя файла (string, до 120 Байт)
 - storagePath - путь или ключ файла во внешнем хранилище (string, до 200 Байт)
 - mimeType - MIME-тип файла (string, до 60 Байт)
 - sizeBytes - размер файла в байтах (int64, 8 Байт)
 - checksumSha256 - контрольная сумма файла (string, 64 Байта)
 - rowCount - число строк, если файл табличный (int32 | null, 4 Байта)
 - status - статус обработки конкретного файла (string, до 12 Байт)
 - createdAt - дата регистрации файла (date, 8 Байт)
 - updateTime - дата последнего изменения метаданных файла (date, 8 Байт)
 - metadata - дополнительные метаданные файла (object)
- statusHistory - история смены статусов загрузки, содержит элементы:
 - oldStatus - предыдущий статус (string, до 12 Байт)
 - newStatus - новый статус (string, до 12 Байт)
 - changedAt - время изменения статуса (date, 8 Байт)
 - changedBy - пользователь или системный процесс, изменивший статус (objectId | string)
 - reason - причина изменения статуса (string, до 200 Байт)
- processingLog - журнал обработки загрузки, содержит элементы:
 - timestamp - время события (date, 8 Байт)
 - level - уровень записи (string, до 5 Байт): info, warn, error

- sourceFileKey - ключ файла-источника в files (string, до 50 Байт)
- line - номер строки в файле (int32 | null, 4 Байт)
- entityType - тип сущности (string, до 30 Байт): student, timeline_event, ocr_frame
- message - текст сообщения (string, до 200 Байт)
- unresolvedStudents - несопоставленные студенты, содержит элементы:
 - externalId - внешний идентификатор студента (string, до 50 Байт)
 - reason - причина несопоставления (string, до 200 Байт)

Коллекция students содержит справочник студентов с вузовской привязкой и эталонными данными для распознавания лиц.

- _id - уникальный идентификатор студента (objectId, 12 Байт)
- uploadId - идентификатор пачки загрузки (обязательное, objectId, 12 Байт)
- universityId - идентификатор вуза (обязательное, objectId, 12 Байт)
- externalId - идентификатор из внешней системы Moodle (обязательное, string, до 50 Байт)
- recordBookNumber - номер зачетной книжки (string, до 30 Байт)
- fullName - ФИО студента (обязательное, string, до 120 Байт)
- email - электронная почта студента (string, до 120 Байт)
- faculty - факультет или институт (string, до 120 Байт)
- program - образовательная программа (string, до 120 Байт)
- educationLevel - уровень образования (string, до 30 Байт): bachelor, master, specialist
- group - учебная группа (string, до 30 Байт)
- faceEmbedding - вектор эмбединга лица для распознавания (array of double, 512 элементов x 8 Байт = 4096 Байт)
- createdAt - дата создания записи (date, 8 Байт)

- updateTime - дата последнего изменения записи (date, 8 Байт)

Коллекция timeline_events содержит единый таймлайн событий по экзаменационной сессии. В нее попадают Moodle-события, OCR-кадры и системные события. Благодаря общему полю eventTime интерфейс может строить хронологическую ленту без объединения разных коллекций.

- _id - уникальный идентификатор события (objectId, 12 Байт)
- uploadId - идентификатор пачки загрузки (обязательное, objectId, 12 Байт)
- studentId - идентификатор студента (обязательное, objectId, 12 Байт)
- sessionId - идентификатор сессии (objectId | null, 12 Байт)
- eventType - тип события (обязательное, string, до 20 Байт): moodle, ocr_frame, system
- eventTime - абсолютная дата и время события или видеофрейма (обязательное, date, 8 Байт)
- sourceFileKey - ключ исходного файла в uploads.files (string, до 50 Байт)
- moodle - данные Moodle-события, заполняются при eventType = "moodle":
 - action - тип действия (string, до 30 Байт): view, submit, navigate, login, logout
 - target - целевая страница или ресурс (string, до 200 Байт)
 - courseName - название курса (string, до 200 Байт)
 - ip - IP-адрес пользователя (string, до 45 Байт)
 - userAgent - User-Agent браузера (string, до 200 Байт)
 - quizId - идентификатор теста (string, до 30 Байт)
 - questionId - идентификатор вопроса (string, до 30 Байт)
 - answer - данные ответа студента (string, до 500 Байт)
 - isCorrect - правильность ответа (bool, 1 Байт)
 - timeSpent - время на вопрос в секундах (int32, 4 Байта)

- tabFocus - была ли вкладка активной в момент действия (bool, 1 Байт)
- ocr - данные OCR-кадра, заполняются при eventType = "ocr_frame":
 - frameIndex - номер кадра в видео или последовательности кадров (int32, 4 Байта)
 - videoOffsetMs - смещение кадра от начала видео в миллисекундах (int64, 8 Байт)
 - blocks - массив распознанных блоков на кадре:
 - label - тип блока (string, до 30 Байт): text, header, image, paragraph_title, footer, number
 - content - распознанный текст или HTML-разметка (string, до 500 Байт)
 - bbox - ограничивающий прямоугольник [x1, y1, x2, y2] (array of int32, 4 x 4 = 16 Байт)
 - confidence - уверенность распознавания (double, 8 Байт)
 - order - порядковый номер блока на странице (int32 | null, 4 Байта)
 - markdown - итоговый markdown-текст кадра (string, до 1000 Байт)
- system - данные системного события, заполняются при eventType = "system":
 - code - код события (string, до 50 Байт)
 - message - описание события (string, до 200 Байт)
- createdAt - дата создания события в системе (date, 8 Байт)
- updateTime - дата последнего изменения события (date, 8 Байт)

Коллекция sessions содержит собранные экзаменационные сессии с предвычисленными метриками и вектором признаков для кластеризации.

- _id - уникальный идентификатор сессии (objectId, 12 Байт)
- uploadId - идентификатор пачки загрузки (обязательное, objectId, 12 Байт)
- studentId - идентификатор студента (обязательное, objectId, 12 Байт)
- startTime - время начала сессии (обязательное, date, 8 Байт)
- endTime - время окончания сессии (обязательное, date, 8 Байт)
- durationMinutes - длительность сессии в минутах (double, 8 Байт)
- examName - название экзамена (string, до 200 Байт)
- metrics - предвычисленные метрики сессии:
 - moodle - метрики на основе Moodle-событий:
 - totalActions - общее количество действий (int32, 4 Байта)
 - actionsPerMinute - частота активности, действий в минуту (double, 8 Байт)
 - viewToSubmitRatio - отношение просмотров к ответам (double, 8 Байт)
 - pageTransitions - количество переходов между страницами (int32, 4 Байта)
 - uniquePagesVisited - количество уникальных посещенных страниц (int32, 4 Байта)
 - avgTimePerQuestion - среднее время на вопрос в секундах (double, 8 Байт)
 - tabSwitchCount - количество переключений вкладок (int32, 4 Байта)
 - correctAnswerRate - доля правильных ответов (double, 8 Байт)
 - idleIntervals - количество простоев более 2 минут (int32, 4 Байта)

- pasteEventCount - количество вставок из буфера обмена (int32, 4 Байта)
- ocr - метрики на основе OCR-анализа:
 - totalFramesAnalyzed - всего проанализированных кадров (int32, 4 Байта)
 - faceAbsenceRate - доля времени без лица в кадре (double, 8 Байт)
 - foreignFaceRate - доля времени с чужим лицом (double, 8 Байт)
 - faceSwapCount - число смен лиц в кадре (int32, 4 Байта)
 - avgFaceConfidence - средняя уверенность распознавания лица (double, 8 Байт)
 - suspiciousTextDetected - число кадров с подозрительным текстом (int32, 4 Байта)
 - screenContentChanges - число резких изменений содержимого экрана (int32, 4 Байта)
 - avgBlocksPerFrame - среднее количество OCR-блоков на кадр (double, 8 Байт)
- combined - комбинированные метрики:
 - anomalyScore - общий балл аномальности сессии от 0 до 1 (double, 8 Байт)
 - riskLevel - уровень риска (string, до 10 Байт): low, medium, high
- featureVector - вектор числовых признаков для алгоритмов кластеризации (array of double, F элементов x 8 Байт)
- createdAt - дата создания сессии (date, 8 Байт)
- updateTime - дата последнего изменения сессии (date, 8 Байт)

Коллекция `clustering_runs` содержит результаты запусков кластеризации с привязкой к сессиям. Кластеры не объединяются с сессиями, потому что одна и та же сессия может участвовать в разных запусках с разными параметрами, признаками и алгоритмами.

Коллекция `audit_logs` содержит аудит всех действий пользователей и системных процессов. Записи аудита являются `append-only`: они не редактируются после создания, поэтому отдельное поле `updateTime` для них не требуется.

- `_id` - уникальный идентификатор записи аудита (`objectId`, 12 Байт)
- `actorUserId` - пользователь, выполнивший действие (`objectId` | `null`, 12 Байт)
- `actorType` - тип субъекта действия (`string`, до 20 Байт): `user`, `system`
- `action` - действие (`string`, до 50 Байт): `upload.create`, `upload.status.change`, `session.build`, `cluster.run`, `export.csv`
- `entityType` - тип сущности, над которой выполнено действие (`string`, до 30 Байт)
- `entityId` - идентификатор сущности (`objectId` | `string`)
- `occurredAt` - время действия (`date`, 8 Байт)
- `ip` - IP-адрес пользователя (`string`, до 45 Байт)
- `userAgent` - User-Agent браузера (`string`, до 200 Байт)
- `before` - состояние значимых полей до действия (`object`, примерно 200 Байт)
- `after` - состояние значимых полей после действия (`object`, примерно 200 Байт)
- `details` - дополнительные детали действия (`object`, примерно 300 Байт)

Оценка объёма MongoDB-модели

Введение обозначений

N - количество экзаменационных сессий, основная переменная.

P - количество пользователей системы.

W - количество пачек загрузки.

Q - количество уникальных студентов.

G - среднее количество файлов в одной загрузке.

M - среднее количество Moodle-событий на одну сессию.

R - среднее количество OCR-кадров на одну сессию.

B - среднее количество OCR-блоков на одном кадре.

L - среднее количество записей журнала обработки на одну загрузку.

U - среднее количество несопоставленных студентов на одну загрузку.

H - среднее количество смен статуса на одну загрузку или запуск.

A - среднее количество аудитных записей на одну сессию.

F - размерность вектора признаков для кластеризации.

C - количество запусков кластеризации.

K - среднее количество кластеров в одном запуске.

Связь количества объектов:

$P \sim 5$, поэтому вклад пользователей пренебрежимо мал.

$W = N / 50$, в среднем 50 сессий на загрузку.

$Q = N / 5$, в среднем 5 экзаменов на студента.

$C = N / 200$, один запуск кластеризации на 200 сессий.

Типичные значения параметров: $G = 3$, $M = 80$, $R = 30$, $B = 12$, $L = 200$, $U = 5$, $H = 4$, $A = 5$, $F = 20$, $K = 5$.

Содержимое исходных файлов не включается в расчёт объёма MongoDB, если файлы хранятся во внешнем файловом или объектном хранилище. В расчёт входят только метаданные этих файлов.

Оценка объёма информации

SX - объём данных одной записи в коллекции X.

Коллекция users

$Susers = 12 + 120 + 60 + 120 + 10 + 8 + 8 + 60$ (ключи BSON) = 398 Байт

Коллекция universities

$Suniversities = 12 + 200 + 50 + 30 + 8 + 8 + 60$ (ключи BSON) = 368 Байт

Коллекция uploads

Базовые поля: $12 + 12 + 8 + 8 + 12 + 100$ (ключи) = 152 Байт

Один элемент files: $120 + 200 + 60 + 8 + 64 + 4 + 12 + 8 + 8 + 120 + 100$ (ключи) = 704 Байт

Одна запись statusHistory: $12 + 12 + 8 + 12 + 200 + 50$ (ключи) = 294 Байт

Одна запись processingLog: $8 + 5 + 50 + 4 + 30 + 200 + 40$ (ключи) = 337 Байт

Одна запись unresolvedStudents: $50 + 200 + 25$ (ключи) = 275 Байт

$Suploads = 152 + 704 * G + 294 * H + 337 * L + 275 * U$ Байт

При $G = 3, H = 4, L = 200, U = 5$:

$Suploads = 152 + 2\,112 + 1\,176 + 67\,400 + 1\,375 = 72\,215$ Байт

Коллекция students

$Sstudents = 12 + 12 + 12 + 50 + 30 + 120 + 120 + 120 + 120 + 30 + 30 + 4096$
 $+ 8 + 8 + 85$ (ключи) = 4 853 Байт

Коллекция timeline_events

Общие поля события: $12 + 12 + 12 + 12 + 20 + 8 + 50 + 8 + 8 + 100$ (ключи)
= 242 Байт

Данные Moodle-события: $30 + 200 + 200 + 45 + 200 + 30 + 30 + 500 + 1 + 4$
 $+ 1 + 90$ (ключи) = 1 331 Байт

$Stimeline_moodle = 242 + 1\,331 = 1\,573$ Байт

Данные OCR-кадра без общих метаданных видео: $4 + 8 + 1000 + 60$ (ключи) = 1 072 Байт

Один OCR-блок: $30 + 500 + 16 + 8 + 4 + 40$ (ключи) = 598 Байт

$Stimeline_ocr = 242 + 1\,072 + 598 * B$ Байт

При $B = 12$:

$Stimeline_ocr = 242 + 1\,072 + 7\,176 = 8\,490$ Байт

Коллекция sessions

Метрики Moodle: $4 + 8 + 8 + 4 + 4 + 8 + 4 + 8 + 4 + 4 = 56$ Байт

Метрики OCR: $4 + 8 + 8 + 4 + 8 + 4 + 4 + 8 = 48$ Байт

Метрики combined: $8 + 10 = 18$ Байт

$S_{sessions} = 12 + 12 + 12 + 8 + 8 + 8 + 200 + 56 + 48 + 18 + 8 * F + 8 + 8 + 220$ (ключи) $= 618 + 8F$ Байт

При $F = 20$:

$S_{sessions} = 618 + 160 = 778$ Байт

Коллекция `clustering_runs`

Базовые поля без вложенных массивов: $12 + 12 + 8 + 8 + 8 + 8 + 10 + 20 + 120 + 60 + 4 + 4 + 8 + 150$ (ключи) $= 432$ Байт

Одна запись `statusHistory`: $10 + 10 + 8 + 12 + 200 + 50$ (ключи) $= 290$ Байт

Один элемент `clusters`: $4 + 4 + 8 * F + 200 + 40$ (ключи) $= 248 + 8F$ Байт

При $F = 20$ один элемент `clusters` $= 408$ Байт

Один элемент `sessionAssignments`: $12 + 4 + 1 + 8 + 16 + 50$ (ключи) $= 91$ Байт

$S_{clustering_runs} = 432 + 290 * H + (248 + 8F) * K + 91 * N_{сессий}$ в запуске Байт

При $H = 4$, $K = 5$, $F = 20$, 200 сессий на запуск:

$S_{clustering_runs} = 432 + 1\,160 + 408 * 5 + 91 * 200 = 21\,832$ Байт

Коллекция `audit_logs`

$S_{audit_logs} = 12 + 12 + 20 + 50 + 30 + 12 + 8 + 45 + 200 + 200 + 200 + 300 + 100$ (ключи) $= 1\,189$ Байт

Запросы к модели, с помощью которых реализуются сценарии использования

1. Массовый импорт данных экзамена

Текст запроса: создать пачку загрузки с метаданными файлов, вставить студентов, вставить события таймлайна, обновить статус и записать аудит.

Количество запросов: 5.

Задействованные коллекции: 4 (uploads, students, timeline_events, audit_logs).

```
// Запрос 1: создание пачки загрузки с map файлов
db.uploads.insertOne({
  userId: ObjectId("<userId>"),
  createdAt: new Date(),
  updateTime: new Date(),
  status: "pending",
  files: {
    students_csv: {
      originalName: "students.csv",
      storagePath: "uploads/<uploadId>/students.csv",
      mimeType: "text/csv",
      sizeBytes: NumberLong(18420),
      checksumSha256: "<checksum>",
      rowCount: 48,
      status: "pending",
      createdAt: new Date(),
      updateTime: new Date(),
      metadata: { delimiter: "," }
    },
    screencast_01: {
      originalName: "exam_screen.mp4",
      storagePath: "uploads/<uploadId>/exam_screen.mp4",
      mimeType: "video/mp4",
      sizeBytes: NumberLong(240118940),
      checksumSha256: "<checksum>",
      rowCount: null,
      status: "pending",
      createdAt: new Date(),
      updateTime: new Date(),
      metadata: {
        width: 3420,
        height: 2224,
        fps: 30,
        durationSeconds: 4320,
        videoStartedAt: ISODate("2026-01-20T09:00:00Z")
      }
    }
  }
})
```

```

        }
    }
},
statusHistory: [
    {
        oldStatus: null,
        newStatus: "pending",
        changedAt: new Date(),
        changedBy: ObjectId("<userId>"),
        reason: "Загрузка создана"
    }
],
processingLog: [],
unresolvedStudents: []
});

// Запрос 2: массовая вставка студентов
db.students.insertMany([
    {
        uploadId: ObjectId("<uploadId>"),
        universityId: ObjectId("<universityId>"),
        externalId: "stud_001",
        recordBookNumber: "23-0001",
        fullName: "Иванов П.С.",
        email: "ivanov@u.ru",
        faculty: "ФКТИ",
        program: "Прикладная информатика",
        educationLevel: "bachelor",
        group: "3530901/10101",
        faceEmbedding: [0.023, -0.045],
        createdAt: new Date(),
        updateTime: new Date()
    }
]);

// Запрос 3: массовая вставка событий таймлайна
db.timeline_events.insertMany(timelineEventsArray);

```



```
// Запрос 4: обновление статуса загрузки с записью истории
db.uploads.updateOne(
  { _id: ObjectId("<uploadId>") },
  {
    $set: { status: "done", updateTime: new Date() },
    $push: {
      statusHistory: {
        oldStatus: "processing",
        newStatus: "done",
        changedAt: new Date(),
        changedBy: "system",
        reason: "Импорт завершен"
      }
    }
  }
);
```

```
// Запрос 5: запись аудита
db.audit_logs.insertOne({
  actorUserId: ObjectId("<userId>"),
  actorType: "user",
  action: "upload.create",
  entityType: "upload",
  entityId: ObjectId("<uploadId>"),
  occurredAt: new Date(),
  ip: "192.168.1.42",
  userAgent: "Mozilla/5.0",
  before: null,
  after: { status: "done" },
  details: { fileKeys: ["students_csv", "screencast_01"] }
});
```

2. Построение сессий и вычисление метрик

Текст запроса: агрегировать события таймлайна по студентам, вычислить метрики и создать документы сессий.

Количество запросов: 4.

Задействованные коллекции: 3 (timeline_events, sessions, audit_logs).

```
// Запрос 1: агрегация Moodle-событий для границ сессий и метрик
```

```

db.timeline_events.aggregate([
  {
    $match: {
      uploadId: ObjectId("<uploadId>"),
      eventType: "moodle"
    }
  },
  { $sort: { studentId: 1, eventTime: 1 } },
  {
    $group: {
      _id: "$studentId",
      firstEvent: { $min: "$eventTime" },
      lastEvent: { $max: "$eventTime" },
      totalActions: { $sum: 1 },
      views: { $sum: { $cond: [{ $eq: ["$moodle.action", "view"] }, 1,
0] } } },
      submits: { $sum: { $cond: [{ $eq: ["$moodle.action", "submit"] }, 1,
0] } } },
      tabSwitches: { $sum: { $cond: [{ $eq: ["$moodle.tabFocus", false] },
1, 0] } } },
      correctCount: { $sum: { $cond: [{ $eq: ["$moodle.isCorrect",
true] }, 1, 0] } } },
      uniquePages: { $addToSet: "$moodle.target" },
      avgTimeSpent: { $avg: "$moodle.timeSpent" }
    }
  }
]);

```

// Запрос 2: агрегация OCR-метрик по будущим границам сессий

```

db.timeline_events.aggregate([
  {
    $match: {
      uploadId: ObjectId("<uploadId>"),
      eventType: "ocr_frame"
    }
  },
  {
    $group: {

```

```

        _id: "$studentId",
        totalFrames: { $sum: 1 },
        avgBlocks: { $avg: { $size: "$ocr.blocks" } }
    }
}
]);

// Запрос 3: массовая вставка собранных сессий
db.sessions.insertMany(sessionsWithMetrics);

// Запрос 4: привязка sessionId к событиям таймлайна
db.timeline_events.bulkWrite(
    sessionMappings.map(m => ({
        updateMany: {
            filter: {
                studentId: m.studentId,
                uploadId: m.uploadId,
                eventTime: { $gte: m.start, $lte: m.end }
            },
            update: { $set: { sessionId: m.sessionId, updateTime: new Date() } }
        }
    })))
);

```

3. Просмотр таймлайна сессии

Текст запроса: получить Moodle-, OCR- и системные события одной сессии в хронологическом порядке.

Количество запросов: 1.

Задействованные коллекции: 1 (timeline_events).

```

db.timeline_events.find(
    { sessionId: ObjectId("<sessionId>") },
    {
        eventType: 1,
        eventTime: 1,
        sourceFileKey: 1,
        moodle: 1,
        ocr: 1,
    }
)

```

```

        system: 1
    }
).sort({ eventTime: 1 });

```

4. Кластеризация и сохранение результатов

Текст запроса: получить сессии с фильтрацией по загрузкам и периоду, затем сохранить результат кластеризации.

Количество запросов: 2.

Закдействованные коллекции: 2 (sessions, clustering_runs).

```

// Запрос 1: получение данных для кластеризации
db.sessions.find(
  {
    uploadId: { $in: [ObjectId("..."), ObjectId("...")] },
    startTime: {
      $gte: ISODate("2026-01-01T00:00:00Z"),
      $lte: ISODate("2026-02-01T00:00:00Z")
    }
  },
  { featureVector: 1, studentId: 1, metrics: 1, examName: 1 }
);

// Запрос 2: сохранение результата кластеризации
db.clustering_runs.insertOne({
  userId: ObjectId("<userId>"),
  uploadIds: [ObjectId("...")],
  startedAt: new Date("2026-02-20T14:00:00Z"),
  finishedAt: new Date("2026-02-20T14:02:15Z"),
  createdAt: new Date("2026-02-20T14:00:00Z"),
  updateTime: new Date("2026-02-20T14:02:15Z"),
  status: "done",
  statusHistory: [
    {
      oldStatus: "running",
      newStatus: "done",
      changedAt: new Date("2026-02-20T14:02:15Z"),
      changedBy: "system",
      reason: "Кластеризация завершена"
    }
  ]
});

```

```

],
algorithm: "kmeans",
parameters: {
  k: 5,
  distanceMetric: "euclidean",
  selectedFeatures: ["totalActions", "faceAbsenceRate"]
},
filter: {
  dateFrom: ISODate("2026-01-01"),
  dateTo: ISODate("2026-02-01")
},
results: {
  totalSessions: 200,
  clusterCount: 5,
  silhouetteScore: 0.72,
  clusters: [
    {
      clusterId: 0,
      size: 78,
      centroid: [85.2, 0.02],
      avgMetrics: { moodle: { totalActions: 85 }, ocr:
{ faceAbsenceRate: 0.02 } }
    }
  ],
  sessionAssignments: [
    {
      sessionId: ObjectId("<sessionId>"),
      clusterId: 0,
      isAnomaly: false,
      distanceToCentroid: 1.34,
      reducedCoords: { x: 2.31, y: -0.87 }
    }
  ]
}
});

```

5. Экспорт аномальных сессий

Текст запроса: получить список аномальных сессий с данными студентов и метриками для CSV-экспорта.

Количество запросов: 1.

Задействованные коллекции: 3 (clustering_runs, sessions, students).

```
db.clustering_runs.aggregate([
  { $match: { _id: ObjectId("<runId>") } },
  { $unwind: "$results.sessionAssignments" },
  { $match: { "results.sessionAssignments.isAnomaly": true } },
  {
    $lookup: {
      from: "sessions",
      localField: "results.sessionAssignments.sessionId",
      foreignField: "_id",
      as: "session"
    }
  },
  { $unwind: "$session" },
  {
    $lookup: {
      from: "students",
      localField: "session.studentId",
      foreignField: "_id",
      as: "student"
    }
  },
  { $unwind: "$student" },
  {
    $project: {
      "student.fullName": 1,
      "student.group": 1,
      "student.recordBookNumber": 1,
      "session.examName": 1,
      "session.startTime": 1,
      "session.metrics": 1,
      clusterId: "$results.sessionAssignments.clusterId",
      distanceToCentroid:
"$results.sessionAssignments.distanceToCentroid"
    }
  }
]);
```

6. История загрузок с фильтрацией

Текст запроса: получить список загрузок за период с данными автора, текущим статусом и файлами.

Количество запросов: 1.

Задействованные коллекции: 2 (uploads, users).

```
db.uploads.aggregate([
  {
    $match: {
      createdAt: {
        $gte: ISODate("2026-01-01T00:00:00Z"),
        $lte: ISODate("2026-03-01T00:00:00Z")
      },
      status: "done"
    }
  },
  {
    $lookup: {
      from: "users",
      localField: "userId",
      foreignField: "_id",
      as: "author"
    }
  },
  { $unwind: "$author" },
  {
    $project: {
      createdAt: 1,
      updateTime: 1,
      status: 1,
      files: 1,
      statusHistory: 1,
      "author.fullName": 1
    }
  },
  { $sort: { createdAt: -1 } }
]);
```

7. Аудит действий

Текст запроса: получить историю действий по конкретной сущности.

Количество запросов: 1.

Задействованные коллекции: 1 (audit_logs).

```
db.audit_logs.find(  
  {  
    entityType: "upload",  
    entityId: ObjectId("<uploadId>")  
  },  
  {  
    actorUserId: 1,  
    actorType: 1,  
    action: 1,  
    occurredAt: 1,  
    before: 1,  
    after: 1,  
    details: 1  
  }  
) .sort({ occurredAt: 1 });
```

2.2. Реляционная СУБД

Графическое представление

- password_hash -- хеш пароля;
- full_name -- ФИО;
- role -- роль пользователя: admin или teacher;
- created_at -- дата создания.

Таблица uploads

Хранит информацию о пачках загрузки данных.

Основные поля:

- id -- первичный ключ;
- user_id -- пользователь, выполнивший загрузку;
- created_at -- дата загрузки;
- status -- статус загрузки;
- files_count -- количество файлов;
- total_rows -- количество строк;
- matched_students -- количество сопоставленных студентов.

Таблица processing_logs

Хранит журнал обработки загрузки.

Основные поля:

- id -- первичный ключ;
- upload_id -- загрузка;
- timestamp -- время события;
- level -- уровень сообщения: info, warn, error;
- source -- источник сообщения;
- line -- номер строки в файле;
- entity_type -- тип обрабатываемой сущности;
- message -- текст сообщения.

Таблица unmatched_students

Хранит студентов, которых не удалось сопоставить при обработке.

Основные поля:

- id -- первичный ключ;
- upload_id -- загрузка;

- external_id -- внешний ID студента;
- reason -- причина ошибки сопоставления.

Таблица students

Хранит студентов.

Основные поля:

- id -- первичный ключ;
- upload_id -- загрузка, из которой пришёл студент;
- external_id -- внешний ID;
- full_name -- ФИО;
- email -- email;
- grp -- учебная группа;
- created_at -- дата создания.

Таблица face_embeddings

Хранит embedding лица студента.

Основные поля:

- id -- первичный ключ;
- student_id -- студент;
- dimension -- номер измерения;
- value -- значение координаты.

Таблица sessions

Хранит экзаменационные сессии студентов.

Основные поля:

- id -- первичный ключ;
- upload_id -- загрузка;
- student_id -- студент;
- start_time, end_time -- границы сессии;
- duration_minutes -- длительность;
- exam_name -- название экзамена.

Moodle-метрики:

- m_total_actions;

- m_actions_per_minute;
- m_view_to_submit_ratio;
- m_page_transitions;
- m_unique_pages_visited;
- m_avg_time_per_question;
- m_tab_switch_count;
- m_correct_answer_rate;
- m_idle_intervals;
- m_paste_event_count.

OCR-метрики:

- o_total_frames_analyzed;
- o_face_absence_rate;
- o_foreign_face_rate;
- o_face_swap_count;
- o_avg_face_confidence;
- o_suspicious_text_detected;
- o_screen_content_changes;
- o_avg_blocks_per_frame.

Комбинированные метрики:

- c_anomaly_score;
- c_risk_level.

Таблица feature_vectors

Хранит векторы признаков сессий.

Основные поля:

- id -- первичный ключ;
- session_id -- сессия;
- dimension -- номер признака;
- value -- значение признака.

Таблица moodle_logs

Хранит события Moodle.

Основные поля:

- id -- первичный ключ;
- upload_id -- загрузка;
- student_id -- студент;
- session_id -- сессия;
- timestamp -- время события;
- action -- действие;
- target -- объект действия;
- course_name -- курс;
- ip -- IP-адрес;
- user_agent -- браузер;
- quiz_id -- тест;
- question_id -- вопрос;
- answer -- ответ;
- is_correct -- корректность ответа;
- time_spent -- время на вопрос;
- tab_focus -- был ли фокус на вкладке.

Таблица ocr_results

Хранит результаты OCR/распознавания экрана.

Основные поля:

- id -- первичный ключ;
- upload_id -- загрузка;
- student_id -- студент;
- session_id -- сессия;
- frame_timestamp -- время кадра;
- page_width, page_height -- размер кадра;
- markdown -- распознанный текст в markdown.

Таблица ocr_blocks

Хранит отдельные OCR-блоки внутри кадра.

Основные поля:

- id -- первичный ключ;
- ocr_result_id -- OCR-кадр;
- label -- тип блока;
- content -- текст;
- bbox_x1, bbox_y1, bbox_x2, bbox_y2 -- координаты;
- confidence -- уверенность распознавания;
- block_order -- порядок блока.

Таблица clustering_runs

Хранит запуски кластеризации.

Основные поля:

- id -- первичный ключ;
- user_id -- пользователь;
- started_at, finished_at -- время запуска;
- status -- статус;
- algorithm -- алгоритм;
- param_k -- число кластеров;
- param_eps -- epsilon для DBSCAN;
- param_min_samples -- min samples для DBSCAN;
- param_distance_metric -- метрика расстояния;
- filter_date_from, filter_date_to -- фильтр по датам;
- total_sessions -- число сессий;
- cluster_count -- число кластеров;
- anomaly_count -- число аномалий;
- anomaly_rate -- доля аномалий;
- silhouette_score -- качество кластеризации.

Таблица run_uploads

Связывает запуск кластеризации с загрузками.

Основные поля:

- run_id -- запуск кластеризации;
- upload_id -- загрузка.

Таблица `selected_features`

Хранит признаки, выбранные для кластеризации.

Основные поля:

- `id` -- первичный ключ;
- `run_id` -- запуск;
- `feature_name` -- название признака.

Таблица `clusters`

Хранит кластеры внутри запуска.

Основные поля:

- `id` -- первичный ключ;
- `run_id` -- запуск;
- `cluster_num` -- номер кластера;
- `size` -- размер;
- `anomaly_rate` -- доля аномалий.

Таблица `cluster_centroids`

Хранит центроиды кластеров.

Основные поля:

- `id` -- первичный ключ;
- `cluster_id` -- кластер;
- `dimension` -- номер измерения;
- `value` -- значение координаты.

Таблица `session_assignments`

Хранит принадлежность сессий к кластерам.

Основные поля:

- `id` -- первичный ключ;
- `run_id` -- запуск;
- `session_id` -- сессия;
- `cluster_id` -- кластер;
- `is_anomaly` -- является ли сессия аномальной;
- `distance_to_centroid` -- расстояние до центроида;

reduced_x, reduced_y -- координаты для 2D-визуализации.

Оценка sql модели

Для оценки объёма данных в SQL-модели введём следующие обозначения:

- N -- количество экзаменационных сессий
- P -- количество пользователей
- Q -- количество загрузок
- S -- количество студентов
- M -- среднее количество Moodle-событий на одну сессию
- R -- среднее количество OCR-кадров на одну сессию
- B -- среднее количество OCR-блоков на один OCR-кадр
- L -- среднее количество записей журнала обработки на одну загрузку
- U -- среднее количество несопоставленных студентов на одну загрузку
- E -- размер embedding-вектора лица студента
- F -- размер feature vector одной сессии
- K -- среднее количество кластеров в одном запуске
- C -- среднее количество запусков кластеризации
- A -- среднее количество выбранных признаков в одном запуске

Для упрощения оценки введём усреднённые коэффициенты:

- $P = 5$ -- фиксированное количество пользователей
- $Q = N / 50$ -- одна загрузка примерно на 50 сессий
- $S = N / 5$ -- один студент примерно на 5 сессий
- $M = 80$ -- 80 Moodle-событий на одну сессию
- $R = 30$ -- 30 OCR-кадров на одну сессию
- $B = 12$ -- 12 OCR-блоков на один OCR-кадр
- $L = 200$ -- 200 записей журнала обработки на одну загрузку
- $U = 5$ -- 5 несопоставленных студентов на одну загрузку
- $E = 512$ -- 512 координат face embedding на одного студента
- $F = 20$ -- 20 признаков в feature vector одной сессии
- $K = 5$ -- 5 кластеров в одном запуске

- $C = N / 200$ -- один запуск кластеризации примерно на 200 сессий
- $A = 5$ -- 5 выбранных признаков в одном запуске

Тогда:

$$S = N / 5$$

$$Q = N / 50$$

$$C = N / 200$$

Количество строк в таблицах, зависящих от вложенных массивов:

$$\text{face_embeddings} = S * E = N / 5 * 512 = 102.4N$$

$$\text{feature_vectors} = N * F = 20N$$

$$\text{moodle_logs} = M * N = 80N$$

$$\text{ocr_results} = R * N = 30N$$

$$\text{ocr_blocks} = R * B * N = 30 * 12 * N = 360N$$

$$\text{clusters} = C * K = N / 200 * 5 = 0.025N$$

$$\text{cluster_centroids} = C * K * F = N / 200 * 5 * 20 = 0.5N$$

$$\text{selected_features} = C * A = N / 200 * 5 = 0.025N$$

$$\text{session_assignments} = N$$

Заголовок строки PostgreSQL:

$H = 23$ Байта на каждую строку.

Оценка объема информации

SX - объём данных одной строки в таблице X (включая заголовок строки 23 Байта).

Таблица users

$$S_{\text{users}} = 4 + 120 + 60 + 120 + 10 + 8 + H$$

$$S_{\text{users}} = 4 + 120 + 60 + 120 + 10 + 8 + 23 = 345 \text{ Байт}$$

Таблица uploads

$$S_{\text{uploads}} = 4 + 4 + 8 + 20 + 4 + 4 + 4 + H$$

$$S_{\text{uploads}} = 4 + 4 + 8 + 20 + 4 + 4 + 4 + 23 = 71 \text{ Байт}$$

Таблица processing_logs

$$S_{\text{processing_logs}} = 4 + 4 + 8 + 10 + 100 + 4 + 30 + 200 + H$$

$$S_{\text{processing_logs}} = 4 + 4 + 8 + 10 + 100 + 4 + 30 + 200 + 23 = 383 \text{ Байт}$$

Таблица unmatched_students

$$\text{Sunmatched_students} = 4 + 4 + 50 + 200 + H$$

$$\text{Sunmatched_students} = 4 + 4 + 50 + 200 + 23 = 281 \text{ Байт}$$

Таблица students (без эмбединга)

$$\text{Sstudents} = 4 + 4 + 50 + 120 + 120 + 30 + 8 + H$$

$$\text{Sstudents} = 4 + 4 + 50 + 120 + 120 + 30 + 8 + 23 = 359 \text{ Байт}$$

Таблица face_embeddings (512 строк на студента)

$$\text{Sface_embeddings} = 4 + 4 + 4 + 8 + H$$

$$\text{Sface_embeddings} = 4 + 4 + 4 + 8 + 23 = 43 \text{ Байт}$$

Таблица sessions

Метрики Moodle:

$$4 * 6 + 8 * 4 = 56 \text{ Байт}$$

Метрики OCR:

$$4 * 4 + 8 * 4 = 48 \text{ Байт}$$

Метрики combined:

$$8 + 10 = 18 \text{ Байт}$$

$$\text{Ssessions} = 4 + 4 + 4 + 8 + 8 + 8 + 200 + 56 + 48 + 18 + H$$

$$\text{Ssessions} = 4 + 4 + 4 + 8 + 8 + 8 + 200 + 56 + 48 + 18 + 23 = 381 \text{ Байт}$$

Таблица feature_vectors (20 строк на сессию)

$$\text{Sfeature_vectors} = 4 + 4 + 4 + 8 + H$$

$$\text{Sfeature_vectors} = 4 + 4 + 4 + 8 + 23 = 43 \text{ Байт}$$

Таблица moodle_logs

$$\text{Smoodle_logs} = 4 + 4 + 4 + 4 + 8 + 30 + 200 + 200 + 45 + 200 + 30 + 30 + 500 + 1 + 4 + 1 + H$$

$$\text{Smoodle_logs} = 4 + 4 + 4 + 4 + 8 + 30 + 200 + 200 + 45 + 200 + 30 + 30 + 500 + 1 + 4 + 1 + 23 = 1\,288 \text{ Байт}$$

Таблица ocr_results (без блоков)

$$\text{Socr_results} = 4 + 4 + 4 + 4 + 8 + 4 + 4 + 1000 + H$$

$$\text{Socr_results} = 4 + 4 + 4 + 4 + 8 + 4 + 4 + 1000 + 23 = 1\,055 \text{ Байт}$$

Таблица ocr_blocks (12 строк на кадр)

$$Socr_blocks = 4 + 4 + 30 + 500 + 16 + 8 + 4 + H$$

$$Socr_blocks = 4 + 4 + 30 + 500 + 16 + 8 + 4 + 23 = 589 \text{ Байт}$$

Таблица clustering_runs

$$Sclustering_runs = 4 + 4 + 8 + 8 + 20 + 30 + 4 + 8 + 4 + 30 + 8 + 8 + 4 + 4 + 4 + 8 + 8 + H$$

$$Sclustering_runs = 4 + 4 + 8 + 8 + 20 + 30 + 4 + 8 + 4 + 30 + 8 + 8 + 4 + 4 + 4 + 8 + 8 + 23 = 187 \text{ Байт}$$

Таблица clusters

$$Sclusters = 4 + 4 + 4 + 4 + 8 + H$$

$$Sclusters = 4 + 4 + 4 + 4 + 8 + 23 = 47 \text{ Байт}$$

Таблица cluster_centroids (F строк на кластер)

$$Scluster_centroids = 4 + 4 + 4 + 8 + H$$

$$Scluster_centroids = 4 + 4 + 4 + 8 + 23 = 43 \text{ Байт}$$

Таблица session_assignments

$$Ssession_assignments = 4 + 4 + 4 + 4 + 1 + 8 + 8 + 8 + H$$

$$Ssession_assignments = 4 + 4 + 4 + 4 + 1 + 8 + 8 + 8 + 23 = 64 \text{ Байт}$$

Таблица run_uploads

$$Srun_uploads = 4 + 4 + H$$

$$Srun_uploads = 4 + 4 + 23 = 31 \text{ Байт}$$

Таблица selected_features

$$Sselected_features = 4 + 4 + 50 + H$$

$$Sselected_features = 4 + 4 + 50 + 23 = 81 \text{ Байт}$$

Суммарный объём данных

$$V = Vusers + Vuploads + \dots + Vselected_features$$

$$V = 1\,725 + (1.42 + 1\,532 + 28.1 + 71.8 + 4\,403.2 + 381 + 860 + 103\,040 + 31\,650 + 212\,040 + 0.935 + 0.155 + 1.175 + 21.5 + 64 + 2.025) * N$$

$$V = 1\,725 + 354\,097.31 * N \text{ Байт}$$

Итоговая формула объёма данных от одной переменной

Округлим коэффициент:

$$V_{sql}(N) = 1\,725 + 354\,097 * N \text{ Байт}$$

- ~346 КБ на сессию
- ~346 МБ на 1 000 сессий
- ~3.4 ГБ на 10 000 сессий
- ~34 ГБ на 100 000 сессий

Примеры запросов

1. Массовый импорт данных экзамена

```
INSERT INTO uploads (  
    user_id,  
    created_at,  
    status,  
    files_count,  
    total_rows,  
    matched_students  
)  
VALUES (  
    1,  
    NOW(),  
    'pending',  
    4,  
    0,  
    0  
)  
RETURNING id;  
-- Запрос 2: массовая вставка студентов  
INSERT INTO students (  
    upload_id,  
    external_id,  
    full_name,  
    email,  
    grp,  
    created_at  
)  
VALUES  
    (1, 'stud_001', 'Иванов П.С.', 'ivanov@u.ru', '3530901/10101', NOW()),
```

```

        (1, 'stud_002', 'Петров А.А.', 'petrov@u.ru', '3530901/10101', NOW());
-- Запрос 3: вставка embedding-ов лиц
INSERT INTO face_embeddings (
    student_id,
    dimension,
    value
)
VALUES
    (1, 0, 0.023),
    (1, 1, -0.045),
    (1, 2, 0.112),
    (2, 0, 0.031),
    (2, 1, -0.018),
    (2, 2, 0.091);
-- Запрос 4: массовая вставка Moodle-логов
INSERT INTO moodle_logs (
    upload_id,
    student_id,
    session_id,
    timestamp,
    action,
    target,
    course_name,
    ip,
    user_agent,
    quiz_id,
    question_id,
    answer,
    is_correct,
    time_spent,
    tab_focus
)
VALUES
    (1, 1, NULL, '2026-02-20 10:00:00', 'login', 'exam', 'NoSQL',
    '10.0.0.1', 'Chrome', 'quiz_1', NULL, NULL, NULL, NULL, TRUE),
    (1, 1, NULL, '2026-02-20 10:03:00', 'view', 'question_1', 'NoSQL',
    '10.0.0.1', 'Chrome', 'quiz_1', 'q_1', NULL, NULL, 120, TRUE),

```

```

        (1, 1, NULL, '2026-02-20 10:05:00', 'submit', 'question_1', 'NoSQL',
        '10.0.0.1', 'Chrome', 'quiz_1', 'q_1', 'A', TRUE, 60, TRUE);
-- Запрос 5: вставка OCR-результатов
INSERT INTO ocr_results (
    upload_id,
    student_id,
    session_id,
    frame_timestamp,
    page_width,
    page_height,
    markdown
)
VALUES
    (1, 1, NULL, '2026-02-20 10:03:10', 1920, 1080, 'Вопрос 1. Выберите
    правильный ответ.')
RETURNING id;
-- Запрос 6: вставка OCR-блоков
INSERT INTO ocr_blocks (
    ocr_result_id,
    label,
    content,
    bbox_x1,
    bbox_y1,
    bbox_x2,
    bbox_y2,
    confidence,
    block_order
)
VALUES
    (1, 'header', 'NoSQL. Тест', 100, 40, 600, 80, 0.95, 1),
    (1, 'text', 'Вопрос 1. Выберите правильный ответ.', 120, 150, 900,
210, 0.92, 2);
-- Запрос 7: обновление статуса загрузки и запись в журнал
UPDATE uploads
SET
    status = 'done',
    total_rows = 5240,
    matched_students = 48

```

```
WHERE id = 1;
```

```
INSERT INTO processing_logs (  
    upload_id,  
    timestamp,  
    level,  
    source,  
    line,  
    entity_type,  
    message  
)
```

```
VALUES (  
    1,  
    NOW(),  
    'info',  
    'csv-import',  
    5240,  
    'upload',  
    'Импорт завершён'  
)
```

2. Обработка данных и построение сессий

-- Запрос 1: агрегация Moodle-метрик по студенту

```
SELECT  
    ml.student_id,  
    MIN(ml.timestamp) AS start_time,  
    MAX(ml.timestamp) AS end_time,  
    COUNT(*) AS total_actions,  
    COUNT(*)::DOUBLE PRECISION /  
        NULLIF(EXTRACT(EPOCH FROM (MAX(ml.timestamp) - MIN(ml.timestamp)))  
    / 60, 0)  
        AS actions_per_minute,  
    COUNT(*) FILTER (WHERE ml.action = 'view')::DOUBLE PRECISION /  
        NULLIF(COUNT(*) FILTER (WHERE ml.action = 'submit'), 0)  
        AS view_to_submit_ratio,  
    COUNT(*) FILTER (WHERE ml.action = 'navigate') AS page_transitions,  
    AVG(ml.time_spent) AS avg_time_per_question,  
    COUNT(*) FILTER (WHERE ml.tab_focus = FALSE) AS tab_switch_count,  
    AVG(CASE WHEN ml.is_correct THEN 1 ELSE 0 END) AS correct_answer_rate
```

```

FROM moodle_logs ml
WHERE ml.upload_id = 1
GROUP BY ml.student_id;
-- Запрос 2: агрегация OCR-метрик по студенту
SELECT
    o.student_id,
    COUNT(*) AS total_frames_analyzed,
    AVG(CASE WHEN b.label = 'face_absent' THEN 1 ELSE 0 END) AS
face_absence_rate,
    AVG(CASE WHEN b.label = 'foreign_face' THEN 1 ELSE 0 END) AS
foreign_face_rate,
    COUNT(*) FILTER (WHERE b.label = 'face_swap') AS face_swap_count,
    AVG(b.confidence) AS avg_face_confidence,
    COUNT(*) FILTER (WHERE b.label = 'suspicious_text') AS
suspicious_text_detected,
    COUNT(*) AS screen_content_changes,
    AVG(block_count.blocks_per_frame) AS avg_blocks_per_frame
FROM ocr_results o
LEFT JOIN ocr_blocks b ON b.ocr_result_id = o.id
LEFT JOIN (
    SELECT ocr_result_id, COUNT(*) AS blocks_per_frame
    FROM ocr_blocks
    GROUP BY ocr_result_id
) block_count ON block_count.ocr_result_id = o.id
WHERE o.upload_id = 1
GROUP BY o.student_id;
-- Запрос 3: создание сессий на основе рассчитанных метрик
INSERT INTO sessions (
    upload_id,
    student_id,
    start_time,
    end_time,
    duration_minutes,
    exam_name,
    m_total_actions,
    m_actions_per_minute,
    m_view_to_submit_ratio,
    m_page_transitions,

```



```

m_unique_pages_visited,
m_avg_time_per_question,
m_tab_switch_count,
m_correct_answer_rate,
m_idle_intervals,
m_paste_event_count,
o_total_frames_analyzed,
o_face_absence_rate,
o_foreign_face_rate,
o_face_swap_count,
o_avg_face_confidence,
o_suspicious_text_detected,
o_screen_content_changes,
o_avg_blocks_per_frame,
c_anomaly_score,
c_risk_level
)
VALUES (
    1,
    1,
    '2026-02-20 10:00:00',
    '2026-02-20 11:30:00',
    90,
    'Экзамен по NoSQL',
    140,
    1.55,
    2.1,
    24,
    12,
    48.5,
    3,
    0.82,
    2,
    0,
    360,
    0.04,
    0.01,
    1,

```

```

        0.91,
        0,
        42,
        8.4,
        0.18,
        'low'
    )
RETURNING id;
-- Запрос 4: сохранение feature vector сессии
INSERT INTO feature_vectors (
    session_id,
    dimension,
    value
)
VALUES
    (1, 0, 140),
    (1, 1, 1.55),
    (1, 2, 0.04),
    (1, 3, 0.01),
    (1, 4, 0.18);
-- Запрос 5: привязка Moodle-логов к построенной сессии
UPDATE moodle_logs
SET session_id = 1
WHERE upload_id = 1
    AND student_id = 1
    AND timestamp BETWEEN '2026-02-20 10:00:00' AND '2026-02-20 11:30:00';
-- Запрос 6: привязка OCR-результатов к построенной сессии
UPDATE ocr_results
SET session_id = 1
WHERE upload_id = 1
    AND student_id = 1
    AND frame_timestamp BETWEEN '2026-02-20 10:00:00' AND '2026-02-20
11:30:00';

```

2.3. Сравнение моделей

Сравнение объёма и структуры

Параметр	MongoDB	Реляционная модель
Основная	8 коллекций	~18-20 таблиц

Параметр	MongoDB	Реляционная модель
схема		
Исходные файлы	Map uploads.files с метаданными и путями	Отдельная таблица upload_files
История статусов	Вложенный массив statusHistory	Отдельная таблица status_history
Moodle и OCR	Единая коллекция timeline_events	Базовая таблица событий + таблицы деталей
OCR-блоки	Массив ocr.blocks внутри события	Отдельная таблица ocr_blocks
Векторы	Массивы faceEmbedding, featureVector, centroid	Отдельные таблицы элементов вектора
Результат кластеризации	Один документ запуска	Несколько таблиц внутри транзакции
Аудит	Одна append-only коллекция	Одна таблица аудита

Удельный объём MongoDB-модели по приведенной оценке:

$$V_{\text{mongo}}(N) = 2\,358 + 389\,787 * N \text{ Байт}$$

Для реляционной модели объём без индексов может быть сопоставимым, но при хранении OCR-блоков, эмбедингов лиц, векторов признаков и центроидов в отдельных строках появляется значительная служебная нагрузка: заголовки строк, surrogate keys, FK-столбцы и B-tree индексы. Поэтому при учёте индексов реляционная модель требует большего объёма хранения и большего числа операций записи.

Количество запросов по сценариям

Сценарий	MongoDB	Реляционная модель	Причина разницы
Импорт данных	5	8-10	В реляционной модели файлы, OCR-блоки, векторы и аудит пишутся в разные таблицы
Построение сессий	4	6-8	Требуются JOIN-ы событий с таблицами деталей

Сценарий	MongoDB	Реляционн ая модель	Причина разницы
Просмотр таймлайна	1	2-3	Нужно соединять базовые события с Moodle/OCR-детальями
Кластеризация	2	7-8	Центроиды, признаки, связи загрузок и назначения сессий нормализованы
Экспорт аномальных сессий	1	1-2	В обеих моделях возможен один составной запрос
История загрузок	1	1-2	В реляционной модели файлы и статусы могут требовать JOIN
Аудит действий	1	1	Структура аудита близка в обеих моделях

MongoDB требует меньше запросов в сценариях, где результат естественно является документом или вложенной структурой: импорт, таймлайн, OCR-данные и кластеризация.

Качественное сравнение

Критерий	MongoDB	Реляционная модель
Гибкость схемы	Новые поля метрик, OCR-блоков и файловых метаданных добавляются без миграции	Добавление полей требует миграций и обновления SQL-запросов
Чтение таймлайна	Все события сортируются по eventTime в одной коллекции	Нужно объединять базовые события и таблицы деталей
Хранение исходных файлов	Метаданные лежат рядом с загрузкой, содержимое хранится отдельно	Метаданные выносятся в отдельную таблицу
Аудит и статусы	История статуса может быть вложена в документ, аудит - отдельной коллекцией	Обычно используются отдельные таблицы
Целостность ссылок	Контролируется приложением и индексами MongoDB	Есть строгие FK-ограничения
Аналитика	Aggregation Pipeline, приложение выполняет кластеризацию	SQL, оконные функции, CTE
Масштабирование	Встроенное шардирование коллекций	Обычно требуется партиционирование или

Критерий	MongoDB	Реляционная модель
		расширения

Вывод

Для задачи анализа прокторинга и кластеризации экзаменационных сессий MongoDB является более подходящей основной СУБД.

Причины:

- Единый таймлайн событий. Moodle-действия, OCR-кадры и системные события естественно хранятся в одной коллекции `timeline_events` и сортируются по `eventTime`.
- Гибкое хранение файлов. `uploads.files` позволяет хранить map исходных файлов с метаданными, путями, контрольными суммами и параметрами видео без отдельной таблицы.
- Вложенные OCR-данные. OCR-блоки логично находятся внутри события кадра, потому что почти всегда читаются вместе с ним.
- Векторы и результаты кластеризации. `faceEmbedding`, `featureVector`, `centroid` и `sessionAssignments` хранятся в естественном документном виде без EAV-паттерна.
- История и аудит. `statusHistory` отражает динамику смены статусов, а `audit_logs` покрывает действия пользователей и системы.
- Меньше операций для основных сценариев. Импорт, построение таймлайна и сохранение кластеризации требуют меньше запросов и меньше межтабличных связей.

Реляционная модель выигрывает в строгой ссылочной целостности и развитой SQL-аналитике. Однако для текущей задачи основная аналитика выполняется на стороне приложения, а данные имеют вложенную, разнородную и временную структуру. Поэтому нормализация в PostgreSQL усложняет модель, увеличивает число таблиц и JOIN-ов, но не обеспечивает существенного преимущества для сценариев прокторинга.

3. Разработанное приложение

3.1. Краткое описание

Было реализовано полнофункциональное веб-приложение для анализа данных экзаменационного прокторинга. Оно состоит из 3 Docker-контейнеров.

DB: Mongo DB образ.

Backend: Node.js + Express сервер. Отвечает за бизнес-логику приложения: авторизацию, импорт данных, обработку загруженных файлов, хранение событий, управление сущностями, фильтрацию, агрегацию, запуск кластеризации, экспорт отчётов и резервное копирование.

Frontend: SPA-приложение на React. Реализует пользовательский интерфейс для загрузки данных, просмотра таблиц, фильтрации, обработки, настройки кластеризации, визуализации результатов, построения статистики, экспорта отчётов и управления резервными копиями. Используется роутинг, переиспользуемые компоненты таблиц, фильтров, форм и графиков.

3.2. Используемые технологии

БД: MongoDB 7.0.

Backend: Node.js 22, Express 5, MongoDB Node.js Driver, TypeScript 5.

Frontend: React 19, React Router 7, Recharts, Gravity UI, Tailwind CSS, TypeScript 5, Vite 7.

Для поднятия сервисов используется Docker Compose. Backend и frontend запускаются в контейнерах на базе node:22-alpine, база данных запускается из образа mongo:7.

3.3. Снимки экрана приложения

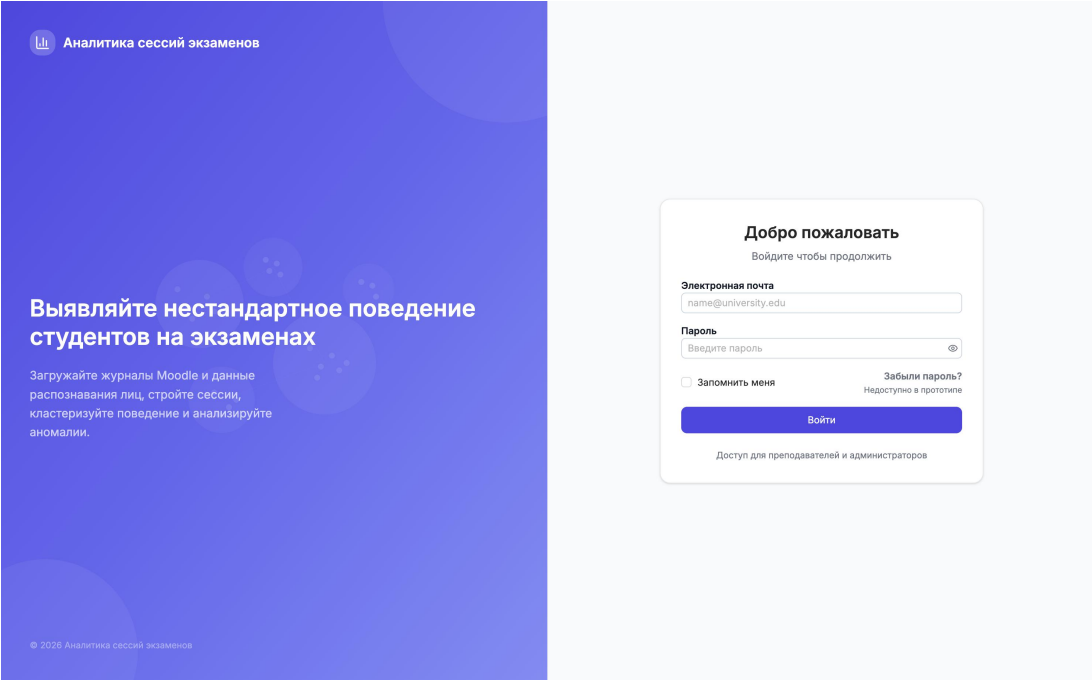


Рисунок 4 – Вход в систему

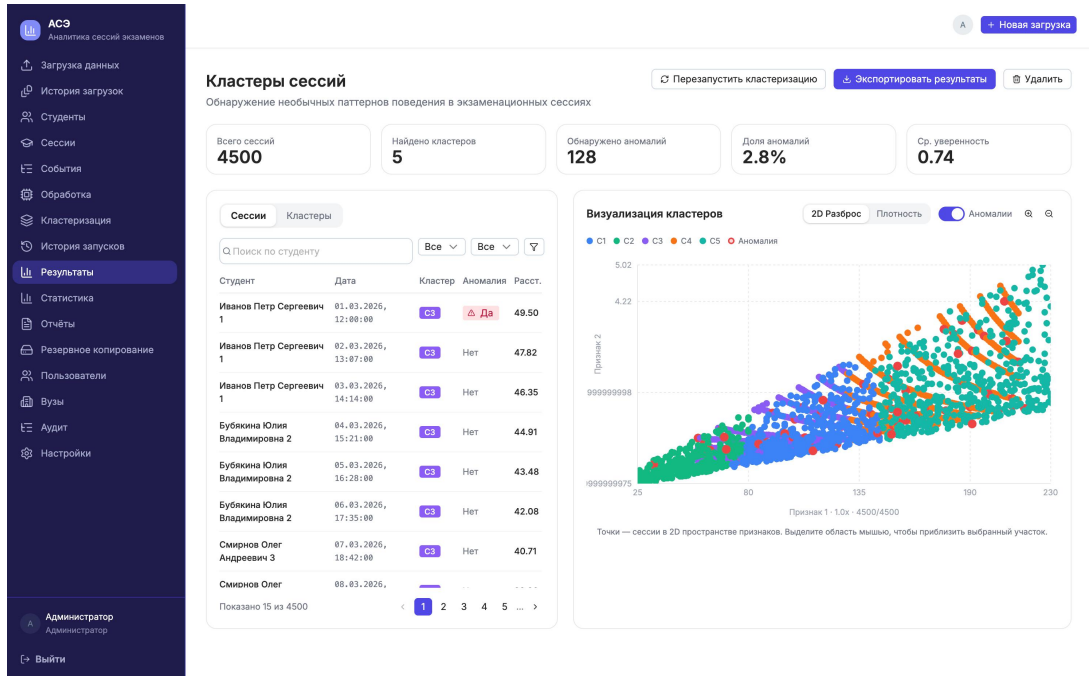


Рисунок 5 – График кластеров сессий

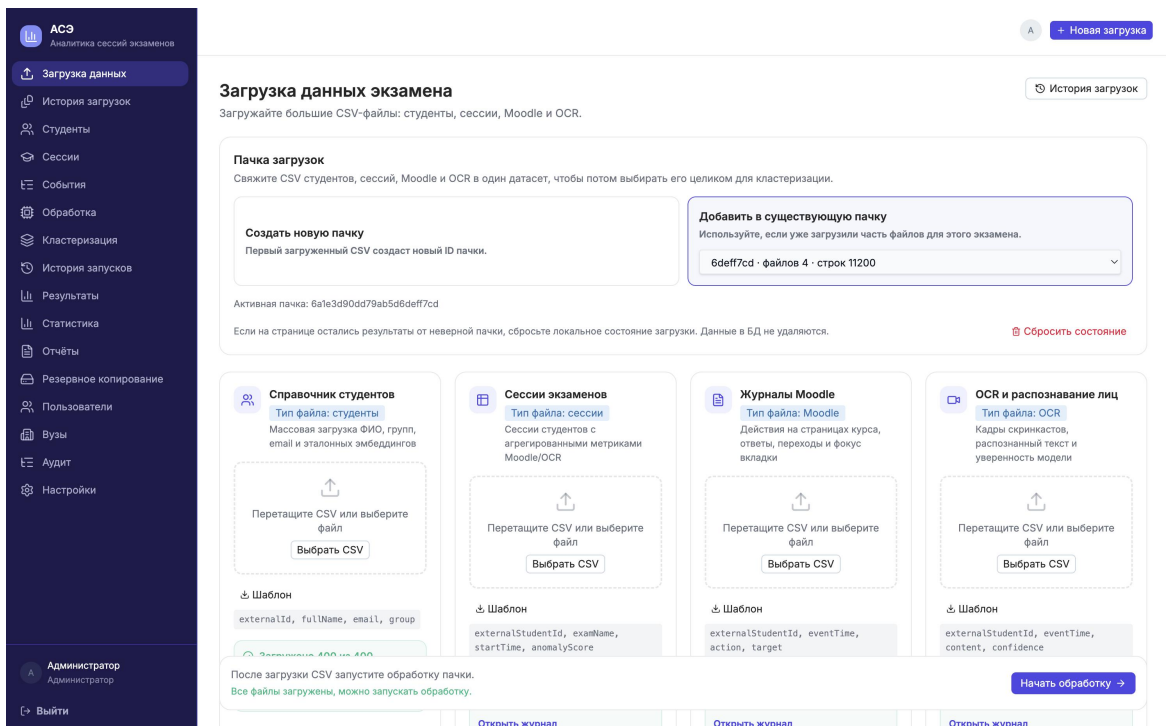


Рисунок 6 – Страница загрузки данных

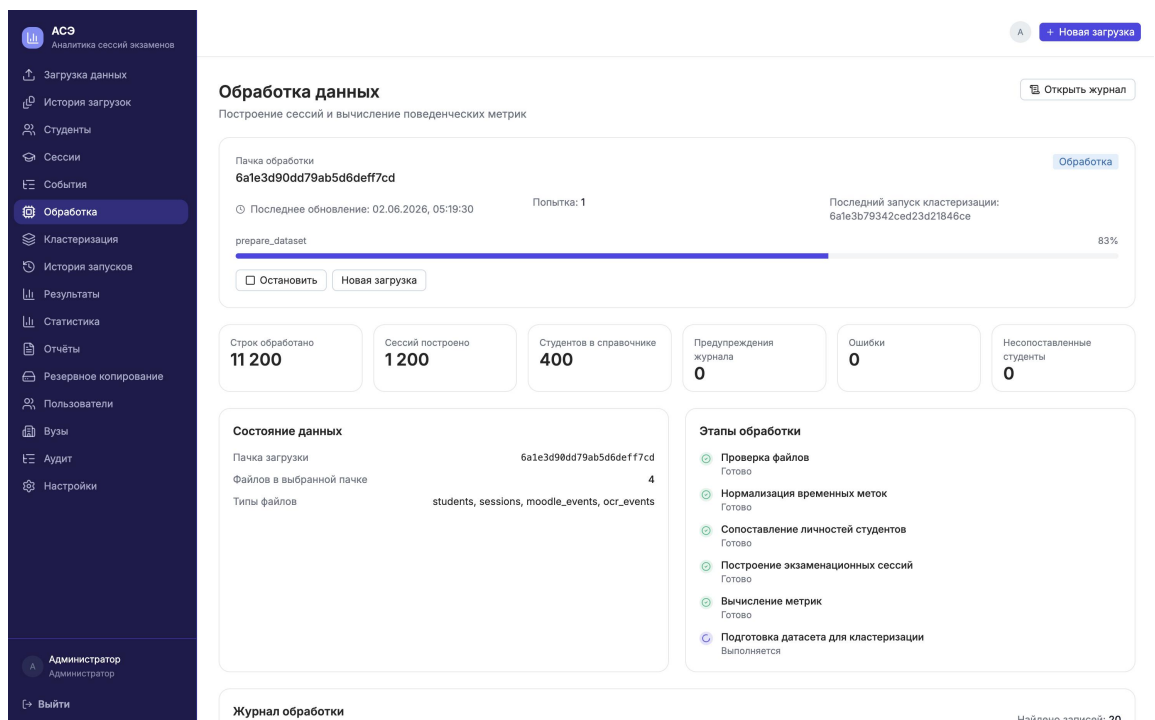


Рисунок 7 – Страница процесса обработки кластеризации

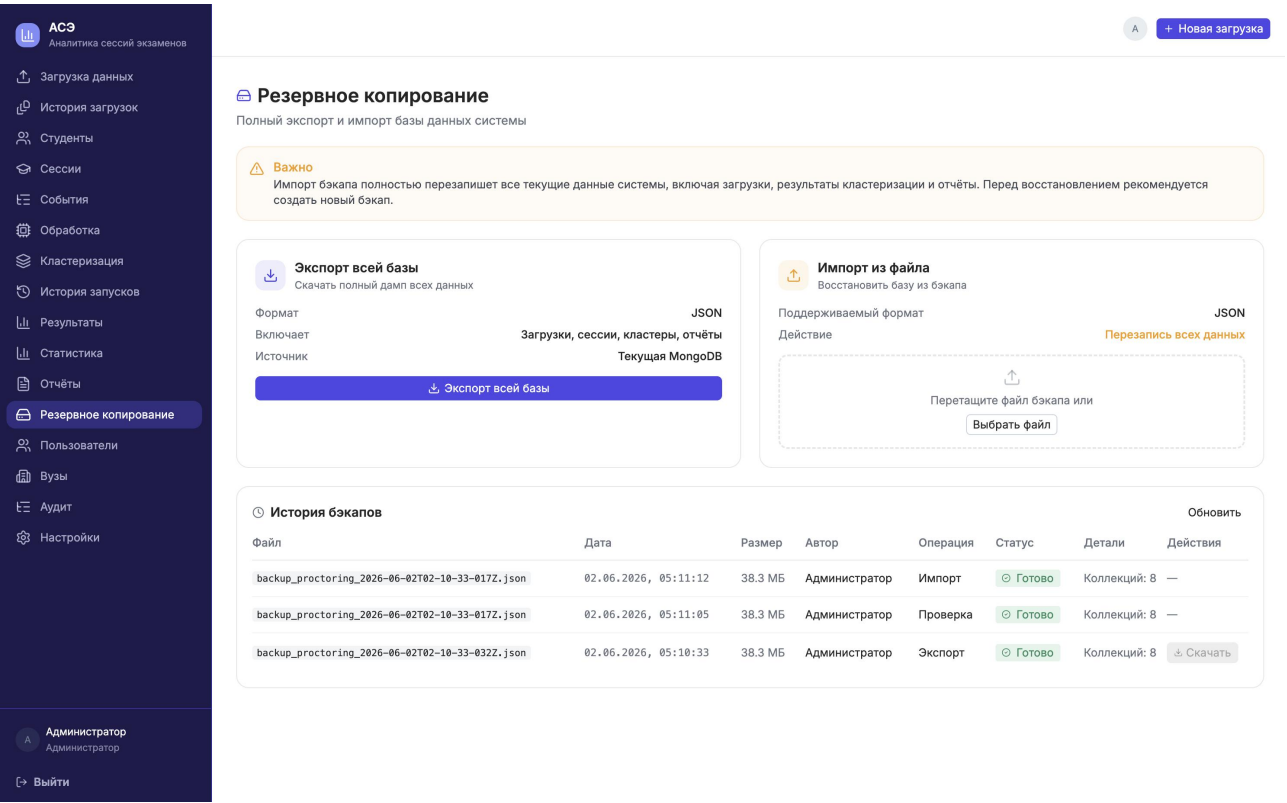


Рисунок 10 - страница импорта и экспорта базы данных

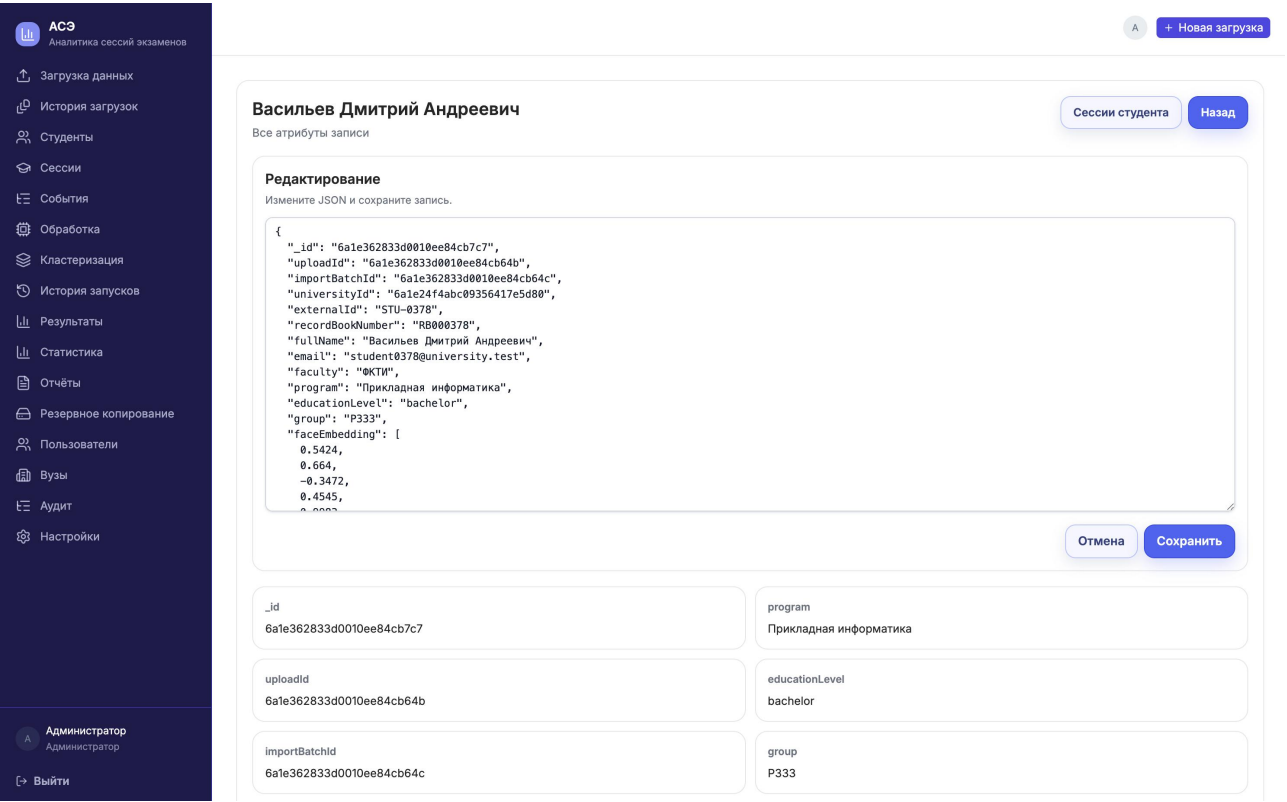


Рисунок 11 - страница отображения и редактирования сущностей приложения

Заключение

Достигнутые результаты

В рамках работы было разработано веб-приложение для хранения и анализа данных экзаменационного прокторинга. Реализованы основные функции системы:

- регистрация и авторизация пользователей;
- загрузка и хранение разнородных данных экзаменационных сессий;
- хранение студентов, сессий, событий, загрузок и результатов кластеризации;
- просмотр данных в табличном виде с фильтрацией и server-side пагинацией;
- обработка входных данных и формирование единого таймлайна событий;
- построение метрик по экзаменационным сессиям;
- запуск алгоритмов кластеризации;
- визуализация кластеров и выявление аномальных сессий;
- формирование кастомной статистики;
- массовый импорт и экспорт данных;
- резервное копирование и восстановление базы данных;
- ведение журнала действий и аудита.

Недостатки и пути для улучшения полученного решения

Текущая версия приложения имеет ряд ограничений. Интерфейс приложения может быть дополнительно оптимизирован для мобильных устройств и небольших экранов. Также отдельные этапы анализа и обработки данных могут быть расширены для более точной работы с реальными прокторинговыми наборами данных.

Для улучшения производительности приложения можно добавить кэширование часто используемых запросов, фоновые очереди обработки файлов, веб-сокеты для отображения прогресса обработки в реальном времени и более гибкую настройку параметров кластеризации.

Будущее развитие решения

Благодаря модульной архитектуре приложение можно расширять дальше. Например, возможно добавить более гибкую систему ролей, дополнительные алгоритмы машинного обучения, расширенную визуализацию таймлайна экзамена, автоматическое формирование рекомендаций по подозрительным сессиям, интеграцию с LMS-системами и ИИ-ассистента для анализа результатов прокторинга.

Список использованных источников

1. Репозиторий программы: <https://github.com/moevm/nsql1h26-proct/>
2. Документация MongoDB: <https://www.mongodb.com/docs/>
3. Документация по TypeScript (ЯП): <https://www.typescriptlang.org/docs/>
4. Документация по NodeJS (Среда выполнения):
<https://nodejs.org/docs/latest-v24.x/api/index.html>
5. Документация по Vite (Сборщик веб сайта): <https://vite.dev/guide/>

Приложение А.

Документация по сборке и развёртывания приложения

Для запуска приложения необходимо наличие установленного Docker и Docker Compose.

Клонирование репозитория

Необходимо клонировать репозиторий проекта и перейти в директорию приложения:

```
git clone https://github.com/moevm/nsql1h26-proct.git
cd nsql1h26-proct
```

Сборка и запуск приложения

Для сборки и запуска всех контейнеров необходимо выполнить команду:

```
docker compose up -d --build
```

Приложение состоит из трёх контейнеров:

db — база данных MongoDB;

backend — серверная часть приложения;

frontend — клиентская часть приложения.

Проверка запуска

После успешного запуска приложение будет доступно по адресу:

```
http://127.0.0.1:5173
```

Backend API доступен по адресу:

```
http://127.0.0.1:18080/api
```

Для проверки состояния backend можно открыть:

```
http://127.0.0.1:18080/api/health
```

Инструкция для пользователя

После открытия приложения пользователь попадает на страницу авторизации.

Для тестирования системы можно использовать следующие учётные записи:

Админ - admin@example.com / password

Преподаватель - teacher@example.com / password

Основные возможности системы

После авторизации пользователь может:

- загружать исходные данные экзаменационных сессий;
- просматривать историю загрузок;
- отслеживать обработку данных;
- просматривать студентов, сессии и события;
- использовать фильтрацию и server-side пагинацию;
- запускать кластеризацию экзаменационных сессий;
- просматривать результаты кластеризации и аномалии;
- формировать кастомную статистику;
- экспортировать аналитические отчёты.

Администратор дополнительно имеет доступ к:

- управлению пользователями;
- просмотру журнала аудита;
- управлению справочником вузов;
- резервному копированию и восстановлению базы данных.